Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б. Н. Ельцина»
ИРИТ-РТФ

ЛАБОРАТОРНАЯ РАБОТА №5

Словарь языка на дереве префиксов

ОТЧЕТ

Студент: Киприн Сергей Андреевич РИМ-250950

Екатеринбург 2026

Задачи

1. Реализовать структуру данных для хранения словаря языка. Функциональные требования: возможности добавления списка слов, добавления 1 слова, проверки, существует ли заданное слово. Нефункциональные требования: потребление памяти строго меньше чем O(N).

2. Оценить: потребление памяти и скорость работы функций в терминах O. Отдельно оценить ведущие множители (к примеру, O(M), M = 33 * N)

Ссылка на репозиторий Github

https://github.com/angst-storm/js-2026

Ход выполнения

Словарь реализован как бор (Trie) - дерево префиксов. Каждый узел хранит детей в Map и флаг isWord, означающий, что путь от корня до этого узла является целым словом. Корень пустой.

Функции:

– Добавление одного слова (add). Проход от корня по символам слова. Если ребра с такой буквой ещё нет - создаётся новый узел. В конце пути ставится isWord = true. Благодаря этому общие начала слов не дублируются: для «кот», «код» и «коты» префикс «ко» существует в дереве один раз.

– Добавление списка (addAll). Вызов add для каждого слова по очереди.

– Проверка существования слова (has). Проход от корня по буквам запроса. Если на каком-то шаге ребёнка нет - слова нет. Если путь пройден целиком, смотрим isWord: так префикс «ко» сам по себе не считается словом, пока его явно не добавили.

```
class TrieNode {
  constructor() {
    this.children = new Map();
    this.isWord = false;
  }
}

export class Dictionary {
  constructor() {
    this.root = new TrieNode();
    this.nodeCount = 1;
    this.wordCount = 0;
  }

  add(word) {
    let node = this.root;

    for (const char of word) {
      if (!node.children.has(char)) {
        node.children.set(char, new TrieNode());
        this.nodeCount += 1;
      }
      node = node.children.get(char);
    }

    if (!node.isWord) {
      node.isWord = true;
      this.wordCount += 1;
    }
  }

  addAll(words) {
    for (const word of words) {
      this.add(word);
    }
  }

  has(word) {
    let node = this.root;

    for (const char of word) {
      if (!node.children.has(char)) {
        return false;
      }
      node = node.children.get(char);
    }

    return node.isWord;
  }
}
```

Оценка сложности по памяти: узлы соответствуют уникальным префиксам, поэтому V (число узлов бора) ≤ N (суммарная длина всех слов, если хранить их отдельно). Если слова делят префиксы, то V < N - памяти меньше, чем при хранении всех строк целиком (O(N)). Итоговая оценка памяти бора: O(V).

Сложность функций: O(L) для добавления слова (L - длина одного слова), O(L) для проверки наличия слова, $O(\Sigma L_i)$ для добавления массива слова.

Ведущие множители: Если бы у каждого узла был массив детей фиксированной длины σ (размер алфавита), память стала бы O(M), где $M = \sigma \cdot V \approx 33 \cdot V$ - за каждый узел платим 33 слота, даже пустых. На одну букву в add/has приходится константная работа с Map (в среднем O(1)), поэтому скорость остаётся O(L) без большого явного множителя вроде $33 \cdot L$.

В index.js добавлен шаг для проверки работы коллекции.

```
58        You, 3 days ago • solved lab4 …
59    // Словарь языка
60    console.log('\n7. Словарь языка (Trie)\n');
61    const dict = new Dictionary();
62    dict.addAll(['кот', 'коты', 'код', 'дом', 'домой']);
63    dict.add('кора');
64    console.log(`слов: ${dict.wordCount}`);
65    for (const word of ['кот', 'код', 'ко', 'кора', 'собака']) {
66      console.log(`has("${word}") → ${dict.has(word)}`);
67    }

Problems  Output  Debug Console  Terminal  Ports

7. Словарь языка (Trie)

слов: 6
has("кот") → true
has("код") → true
has("ко") → false
has("кора") → true
has("собака") → false
angstorm@MacBook-M3-Pro---angstorm js-2025 %
```

Вывод

В ходе работы реализован словарь языка на боре: добавление одного слова, списка слов и проверка наличия слова выполняются за O(L) по длине слова. За счёт хранения общих префиксов один раз потребление памяти составляет O(V) при $V \leq N$ и на типичных данных оказывается меньше, чем при хранении всех строк отдельно.